

FlaskGuard

A Custom Web Application Firewall and Reverse Proxy

Comprehensive Technical and Functional Report

Planning, Design, Architecture, Implementation, and Evaluation

Development Team & Scopes:

Team Member	Project Scope
Saoud Amine	Flask Scaffold + Dashboard UI (HTML/CSS/JS)
Saaf Ayyoub	YAML Configuration, Rate Limiting, HTTP Basic Auth, Dashboard Backend APIs, Regex Detection Engine, Attack Logger, Docker Containerisation
Abdelmoghith El Asraoui	Machine Learning Classifier (LightGBM), Semantic Fallback, IP Strike Tracking, Auto-Ban Logic, Blacklist Persistence
Mohamed Ben Mouh	File Upload Scanner (7 Layers), Double-Extension Detection, HTTPS/TLS Termination
Amhidi Hamza	Reverse Proxy Subsystem, Demo Backend Application, Unit & Integration Test Suite

Repository: github.com/echoenvoy/FlaskGuard

Date: June 2026

Status: Completed – Final Assessment

Contents

1	Project Planning and Execution	5
1.1	Introduction	5
1.2	Team Responsibilities	5
1.3	Phase-by-Phase Execution Plan	6
1.3.1	Phase 1: Foundation and Scaffold (Saoud Amine)	6
1.3.2	Phase 2: Core Firewall and Detection (Saaf Ayoub)	6
1.3.3	Phase 3: Intelligence and Reputation (Abdelmoghith El Asraoui)	6
1.3.4	Phase 4: Content Validation and TLS (Mohamed Ben Mouh)	7
1.3.5	Phase 5: Routing and Verification (Amhidi Hamza)	7
1.4	Dependency Graph	7
1.5	Execution Timeline	8
1.6	System Architecture Overview	8
2	Flask Scaffold and Dashboard UI	10
2.1	Introduction and Project Overview	10
2.1.1	What is FlaskGuard?	10
2.1.2	Five-Part Structure	10
2.1.3	Scope of This Report	11
2.2	Flask Application Scaffold	11
2.2.1	Framework Selection Rationale	11
2.2.2	Application Entry Point – <code>app.py</code>	11
2.2.3	Inspection Pipeline Order	12
2.2.4	Project Directory Structure	12
2.3	Dependencies and Requirements	12
2.3.1	<code>requirements.txt</code> Analysis	12
2.4	Conclusion	13
3	Configuration, Regex Detection, and Deployment	14
3.1	Introduction and System Overview	14
3.1.1	Core Capabilities	14
3.2	Configuration System	14
3.2.1	Design Goals	14
3.2.2	YAML Configuration Schema – <code>config.yml</code>	15
3.2.3	Environment Variable Overrides	16
3.2.4	Configuration Loading Implementation	16
3.3	Rate Limiting with flask-limiter	16
3.3.1	Motivation	16

3.3.2	flask-limiter Integration	16
3.3.3	Rate Limit Configuration Deep Dive	17
3.3.4	Storage Backend for Rate Limit State	17
3.3.5	Rate Limiting and the Admin Dashboard	17
3.4	Admin Authentication – HTTP Basic Auth	17
3.4.1	Design Decision	17
3.5	Regex-Based Attack Detection – <code>security.py</code>	17
3.5.1	Why Regex?	18
3.5.2	Attack Categories Detected	18
3.5.3	Detection Logic – Inspected Input Sources	18
3.5.4	Regex Pattern Library	18
3.5.5	Detection Function Return Value	19
3.5.6	Test Payloads	20
3.6	Attack Logger – <code>attack_logger.py</code>	20
3.6.1	Design Goals	20
3.6.2	Dual-Output Architecture	20
3.6.3	JSON Event Schema	20
3.6.4	Plaintext Security Log Format	20
3.7	Admin Security Dashboard	21
3.7.1	Overview and Design Philosophy	21
3.7.2	Dashboard Features	21
3.7.3	Backend API Endpoints	21
3.7.4	Manual Ban/Unban via curl	21
3.8	Docker Containerisation	22
3.8.1	Rationale for Containerisation	22
3.8.2	Dockerfile Analysis	22
3.8.3	<code>docker-compose.yml</code> Analysis	22
3.8.4	Network Architecture	22
3.8.5	Environment Variable Injection	23
3.8.6	Protecting Any Real Backend	23
3.9	Complete Request Processing Flow	23
3.9.1	Attack Event Lifecycle Summary	24
3.10	Conclusion	24
4	Machine Learning Classifier and IP Reputation	25
4.1	Introduction	25
4.1.1	The Evasion Problem	25
4.1.2	The Repeat Offender Problem	25
4.1.3	Scope	25
4.1.4	Inspection Pipeline Architecture	26
4.2	Machine Learning Classifier – <code>ml_classifier.py</code>	26
4.2.1	Algorithm Evolution: From Naive Bayes to LightGBM	26
4.2.2	The Overfitting Problem and Its Solution	28
4.2.3	Current LightGBM Configuration	29
4.2.4	Feature Extraction: Character N-gram TF-IDF	29
4.2.5	Training Data	31
4.2.6	Model Training Pipeline	31
4.2.7	Model Persistence	32

4.2.8	Prediction API	32
4.2.9	Admin API – ML Endpoints	33
4.2.10	Retraining Workflow	35
4.3	IP Management System – <code>ip_manager.py</code>	35
4.3.1	Overview	35
4.3.2	Strike System	36
4.3.3	Blacklist Data Structure	36
4.3.4	Ban Check in the Inspection Pipeline	36
4.3.5	Ban Expiry Mechanism	37
4.3.6	Manual Ban and Unban	37
4.3.7	Blacklist Persistence	38
4.3.8	Dashboard Integration	38
4.4	Integration – ML Classifier and IP Manager in the Pipeline	38
4.4.1	Full Detection Pipeline with ML	38
4.4.2	Detection Method Attribution	39
4.5	Conclusion	39
5	File Upload Scanner and HTTPS/TLS Termination	40
5.1	Introduction and File Upload Risks	40
5.2	The Seven-Layer File Upload Scanner	40
5.2.1	Layer 1: Filename Sanitisation and Path Traversal	41
5.2.2	Layer 2: Extension Validation	41
5.2.3	Layer 3: Magic Byte Verification	41
5.2.4	Layer 4: Text Content Scanning	41
5.2.5	Layer 5: SVG-Specific Active Content Detection	41
5.2.6	Layer 6: Archive Bomb Mitigation	42
5.2.7	Layer 7: Office Document Macro Detection	42
5.2.8	Double-Extension Detection	42
5.2.9	Scanner Return Value and Integration	42
5.3	HTTPS/TLS Encryption & Nginx Configuration	42
5.3.1	TLS Termination Architecture	43
5.3.2	Nginx TLS Configuration and Security Headers	43
5.3.3	ProxyFix Middleware and Backend Verification	43
6	Reverse Proxy, Demo Backend, and Testing	44
6.1	Introduction	44
6.1.1	Recap of the Series	44
6.1.2	Scope of This Report	44
6.2	Reverse Proxy — <code>proxy.py</code>	45
6.2.1	Role of the Reverse Proxy in the Architecture	45
6.2.2	Request Forwarding Semantics	45
6.2.3	Header Forwarding Strategy	46
6.2.4	Response Header Filtering	46
6.2.5	Error Handling	47
6.2.6	Redirect Handling	47
6.3	Demo Backend Application — <code>demo_backend/</code>	47
6.3.1	Purpose	47
6.3.2	Endpoint Design	47
6.3.3	The <code>/search</code> Endpoint as a Vulnerability Surface	48

6.4	Test Suite	48
6.4.1	Testing Philosophy	48
6.4.2	Unit Tests	48
6.4.3	Integration Tests	49
6.4.4	Administrative API Tests	50
6.4.5	Running the Test Suite	50
6.5	Complete System Architecture	50
6.5.1	End-to-End Request Processing Pipeline	50
6.5.2	Architecture Diagram	52
6.6	Conclusion	52

1

Project Planning and Execution

Introduction

FlaskGuard is an agentic, modular, and educational Web Application Firewall (WAF) and reverse proxy developed to intercept, inspect, and filter HTTP traffic directed at protected backend services. In an era where web applications face an increasingly complex threat landscape, traditional security models often fail due to static rulesets or lack of inline inspection capabilities. FlaskGuard bridges this gap by combining high-performance pattern matching (regular expressions), automated machine learning inference, structured logging, real-time analytics, seven-layer file scanning, and standards-compliant reverse proxying in a containerized environment.

This project was developed by a team of five software engineers, each tasked with owning a specific phase and set of architectural components. The following section details the division of responsibilities, project timelines, development phases, and dependency constraints that governed the system's integration.

Team Responsibilities

Each team member was assigned a defined scope, representing a functional module of the WAF. The division of labor was structured as follows:

Team Member	Core Responsibilities and Deliverables
Saoud Amine	Flask application scaffold initialization, configuration of standard request hooks, and the design and implementation of the Single Page Application (SPA) dashboard UI (HTML, CSS, JS).
Saaf Ayyoub	YAML configuration parsing, per-IP request rate limiting, HTTP Basic Authentication gates for administrative routes, dashboard API backend development, regular-expression-based attack detection, structured logging, and Docker container composition.

Abdelmoghith Asraoui	El	Statistical machine learning classifier development (LightGBM with TF-IDF vectorization), semantic-level vulnerability detection, IP reputation management (strike tracking, automated ban enforcement), and blacklist persistence.
Mohamed Mouh	Ben	Seven-layer file upload scanner (filename, extension, magic bytes, regex content scanning, SVG tags, archive bombs, and office macros), double-extension protection, TLS termination configuration, and ProxyFix header middleware integration.
Amhidi Hamza		Transparent HTTP reverse proxy forwarding, hop-by-hop header stripping, redirect forwarding, a vulnerable backend application for exploit testing, and the complete unit and integration test suite.

Phase-by-Phase Execution Plan

The project was executed in five sequential phases, corresponding to each member's scope. This structure ensured that fundamental components (like the scaffold and routing logic) were established before downstream features (like ML classifiers or reverse proxies) were integrated:

Phase 1: Foundation and Scaffold (Saoud Amine)

- **Flask Scaffold:** Development of the base application structure, setting up package layouts and application entry points.
- **Dashboard UI:** Building the visual interface using HTML5, CSS3, and vanilla JavaScript to render statistics, lists of attacks, and blacklist management.

Phase 2: Core Firewall and Detection (Saaf Ayoub)

- **YAML Config:** Parsing settings from `config.yml` with environment variable overrides.
- **Rate Limiting:** Setting up request throttling using `flask-limiter`.
- **Basic Auth:** Protecting the `/admin` routes with Basic Authentication.
- **Regex Engine:** Writing regular expression patterns for SQL Injection, XSS, Path Traversal, Command Injection, and File Injection.
- **Attack Logger:** Writing attack records to `attacks.json` and `security.log`.
- **Dockerization:** Containerizing the WAF and backend with Docker and Docker Compose.

Phase 3: Intelligence and Reputation (Abdelmoghith El Asraoui)

- **ML Classifier:** Training a LightGBM classifier with character n-grams to detect obfuscated payloads.

- **Semantic Fallback:** Creating heuristic rules to detect NoSQL, LDAP, and template injection attempts.
- **IP Reputation:** Tracking strikes per IP and applying temporary 60-minute bans upon reaching 5 strikes.

Phase 4: Content Validation and TLS (Mohamed Ben Mouh)

- **File Upload Scanner:** Applying a 7-layer scanning pipeline on multipart requests.
- **Double Extension Block:** Preventing execution-bypass tricks (e.g. `malware.php.jpg`).
- **TLS Termination:** Offloading encryption to Nginx and using ProxyFix to restore client IPs.

Phase 5: Routing and Verification (Amhidi Hamza)

- **Reverse Proxy:** Building the forwarding engine in `proxy.py` with proper header and redirect forwarding.
- **Demo Backend:** Creating a target backend application containing a vulnerable search endpoint to demonstrate the WAF's capabilities.
- **Test Suite:** Developing automated unit and integration tests to validate the firewall pipeline.

Dependency Graph

The integration timeline depended on a strict graph of software components, where early-stage primitives unlocked later-stage layers:

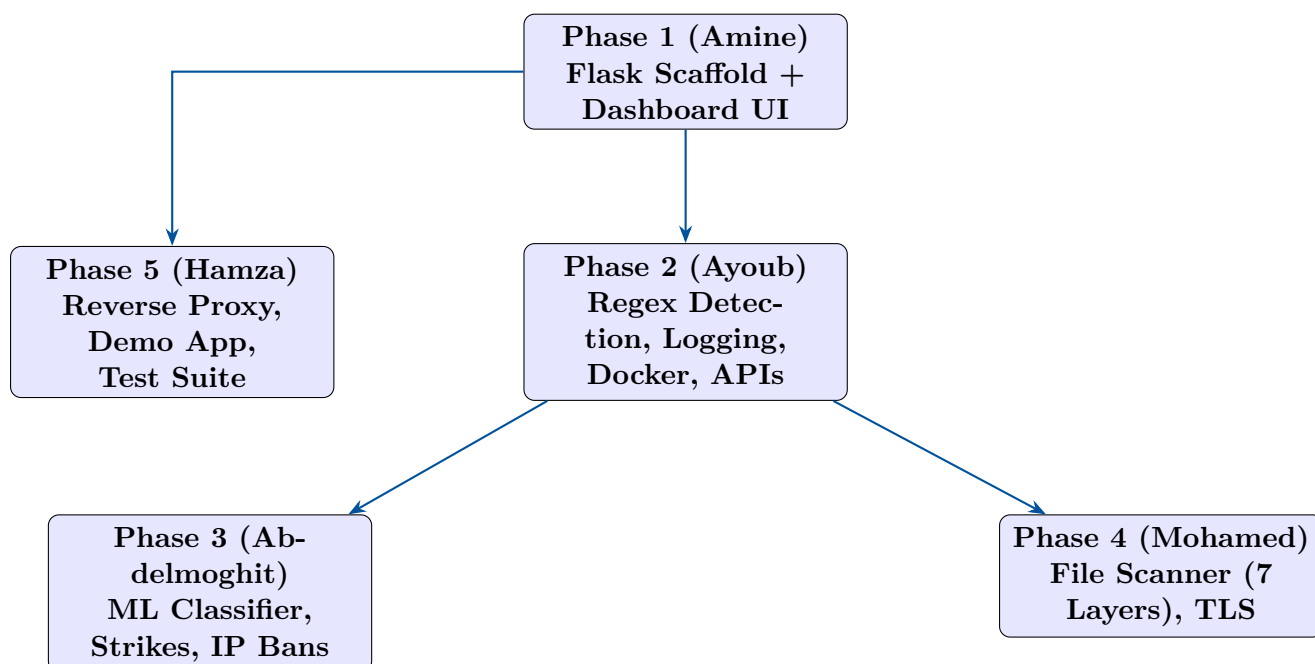


Figure 1.1: FlaskGuard Development Phase Dependencies

Execution Timeline

The project was executed over a ten-week timeline, structured as follows:

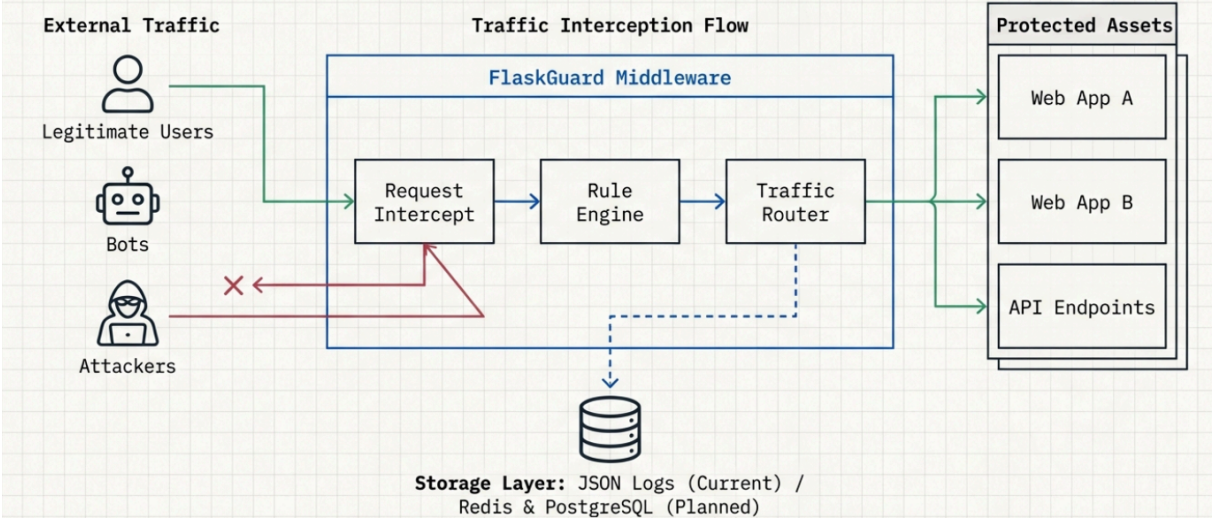
Week	Team Member	Phase	Milestone
Week 1	Amine	Part 1	Flask application scaffold dashboard UI skeleton complete.
Week 2-3	Ayoub	Part 2	Regex engine, logging system, docker environment, and dashboard APIs complete.
Week 4-5	Abdelmoghith	Part 3	ML training pipeline, classification engine, strike logger, and IP blacklist management complete.
Week 6-7	Mohamed	Part 4	7-layer file scanner, double-extension handler, and Nginx TLS config complete.
Week 8-10	Hamza	Part 5	Transparent reverse proxy, vulnerable demo app, and full test suite complete.

System Architecture Overview

The integrated system runs as a multi-container Docker application. When a client initiates an HTTPS request, the traffic flows through the following pipeline:

1. **TLS Termination (Nginx):** Nginx handles certificate negotiation and decodes the HTTPS stream.
2. **Rate Limiting (flask-limiter):** FlaskGuard evaluates the source IP against rate limits. If exceeded, HTTP 429 is returned.
3. **IP Blacklist Check (ip_manager.py):** FlaskGuard checks if the source IP is banned. Banned IPs are blocked immediately (HTTP 403).
4. **File Scanner (file_scanner.py):** If the request is a file upload, it runs through the 7-layer scanner. Malicious files return HTTP 403.
5. **Regex Engine (security.py):** Request parameters are matched against known attack patterns.
6. **ML Classifier (ml_classifier.py):** Clean-appearing requests are evaluated by the LightGBM classifier as a secondary fallback.
7. **Reverse Proxy (proxy.py):** Validated clean requests are forwarded to the internal Docker backend service.

Intercepting Threats Before the Application Layer



2

Flask Scaffold and Dashboard UI

Introduction and Project Overview

What is FlaskGuard?

FlaskGuard is a lightweight, educational Web Application Firewall (WAF) and reverse proxy implemented entirely in Python using the Flask microframework. It is designed to intercept all incoming HTTP traffic directed at a protected backend service, inspect each request for malicious patterns, and either forward the clean request to the backend or block it with an appropriate HTTP error response.

The project demonstrates, in a self-contained and well-documented codebase, the core architectural ideas that underpin commercial WAF products such as Cloudflare, AWS WAF, and ModSecurity. Unlike those enterprise solutions, FlaskGuard is intended for learning, experimentation, controlled internal deployments, and as a portfolio demonstration of applied security engineering.

The complete traffic flow through the system is as follows:

Visitor → **FlaskGuard** → Protected Backend

Five-Part Structure

The project is divided into five development phases:

Part	Scope
1	Flask application scaffold, YAML configuration, rate limiting with <code>flask-limiter</code> , and a custom HTML/JS/CSS Admin Security Dashboard built from scratch.
2	Regex-based attack detection engine, structured attack logger, and Docker containerisation.
3	Machine learning classifier (Naive Bayes with character n-gram TF-IDF), IP management subsystem (strikes, auto-bans, manual ban/unban).
4	File upload scanner (magic bytes, extensions, archives, Office files, SVGs), HTTPS/TLS encryption, and secrets hardening.

Part	Scope
5	Full reverse proxy implementation, demo backend application, and a comprehensive test suite.

Scope of This Report

This document addresses **Part 1** in full technical depth. It covers:

- The Flask application factory and application entry point (`app.py`)
- The YAML configuration schema (`config.yml`) and environment variable overrides
- Rate limiting integration via `flask-limiter`
- The Admin Security Dashboard: both the backend API routes and the custom frontend
- HTTP Basic Authentication protecting the `/admin` namespace
- Design decisions, trade-offs, and recommendations

Flask Application Scaffold

Framework Selection Rationale

Flask was selected as the implementation framework for several well-considered reasons:

- **Minimal surface area:** Flask provides only what is necessary (WSGI adapter, routing, Jinja2 templating, and request/response primitives), keeping the WAF dependency footprint small.
- **Middleware-friendly design:** Flask’s request lifecycle (before-request hooks, after-request hooks, error handlers) maps naturally to the WAF inspection pipeline.
- **WSGI compatibility:** Any WSGI-compatible server (Gunicorn, uWSGI, Waitress) can serve FlaskGuard without code changes.
- **Ecosystem maturity:** Libraries such as `flask-limiter`, `requests` (for the proxy), and `PyYAML` integrate cleanly with Flask without framework-level conflicts.

Application Entry Point – `app.py`

The main application file (`flaskguard/app.py`) serves multiple roles:

1. **Flask instance creation:** Instantiates the Flask application object, configuring template and static file paths.
2. **Configuration loading:** Calls the configuration subsystem to parse `config.yml` and merge environment variable overrides.
3. **Extension initialization:** Sets up `flask-limiter` with the loaded rate-limiting configuration.
4. **Before-request pipeline:** Registers `@app.before_request` hooks to implement the WAF inspection pipeline in order: admin auth check, IP blacklist check, file upload scanning, regex detection, and ML classifier fallback.

5. **Route definitions:** Defines `/admin` and all `/admin/api/*` routes for the dashboard, plus the catch-all `<path:path>` proxy route.
6. **Entry point guard:** A `if __name__ == "__main__"` block starts the development server with the configured port and debug flag.

Inspection Pipeline Order

A critical design decision is the **order** in which security checks are applied. FlaskGuard processes each request through these layers in sequence:

Step	Action
1	Admin Basic Auth check (for <code>/admin</code> namespace)
2	IP blacklist check (immediate 403 for banned IPs)
3	File upload scanner (if <code>Content-Type</code> indicates multipart data)
4	Regex-based attack pattern detection
5	ML classifier (fallback for inputs that pass the regex layer)
6	Forward clean requests to backend via reverse proxy

The order is deliberate: cheap checks (auth, blacklist) run before expensive ones (ML inference), minimising computation for already-blocked IPs. File scanning runs before regex because a malicious filename may not trigger string-based patterns.

Project Directory Structure

The package layout follows Flask application conventions:

```

1 FlaskGuard/
2 |-- flaskguard/
3 |   |-- app.py           # Main Flask app (entry point, routes,
   |   | pipeline)
4 |   |-- security.py      # Regex attack detection
5 |   |-- ml_classifier.py  # Naive Bayes ML model
6 |   |-- file_scanner.py  # Upload scanner
7 |   |-- ip_manager.py    # IP blacklist logic
8 |   |-- attack_logger.py # JSON + file logging
9 |   |-- proxy.py         # Reverse proxy forwarding
10 |   '-- templates/
11 |       '-- dashboard.html # Admin dashboard (all-in-one HTML/CSS/JS)
12 |-- demo_backend/        # Example backend protected by FlaskGuard
13 |-- logs/                # Sample log files
14 |-- models/              # Persisted ML model files
15 |-- config.yml           # Main configuration
16 |-- docker-compose.yml   # WAF + demo backend composition
17 |-- Dockerfile           # Container image definition
18 '-- requirements.txt      # Python dependencies

```

Listing 2.1: FlaskGuard directory tree

Dependencies and Requirements

requirements.txt Analysis

The following is an analysis of the core Part 1 dependencies declared in `requirements.txt`:

Package	Role
<code>flask</code>	Core web framework: routing, request handling, Jinja2 templating
<code>flask-limiter</code>	Per-IP rate limiting with pluggable storage backends
<code>pyyaml</code>	Parsing <code>config.yml</code> into Python dictionaries
<code>requests</code>	HTTP client used by the reverse proxy to forward requests
<code>scikit-learn</code>	ML classifier (Naive Bayes + TF-IDF) – Part 3
<code>joblib</code>	Model serialisation (<code>.pkl</code> files) – Part 3
<code>python-magic</code>	Magic byte detection for file uploads – Part 4

Conclusion

Part 1 of the FlaskGuard project establishes a solid, well-structured foundation for the WAF system. The choice of Flask as the host framework provides a minimal, WSGI-compatible base that integrates cleanly with the rate-limiting, authentication, and dashboard components. The YAML configuration system gives operators comprehensive control over runtime behaviour without requiring code changes, while environment variable overrides support secure container-based deployments following the Twelve-Factor App pattern.

The Admin Security Dashboard demonstrates how a functional, real-time security monitoring interface can be built with vanilla HTML, CSS, and JavaScript, served directly from the Flask application, without any frontend build toolchain. The clean separation between backend API endpoints (`/admin/api/*`) and the dashboard UI (`/admin`) makes the system extensible and testable.

The rate limiting integration provides a first line of defence against brute-force and denial-of-service attacks, and the HTTP Basic Auth gate prevents unauthorised access to the sensitive admin interface. Together, these components form a coherent, deployable foundation that the remaining four parts build upon.

3

Configuration, Regex Detection, and Deployment

Introduction and System Overview

FlaskGuard is a custom Web Application Firewall designed to protect HTTP backends by inspecting incoming requests for malicious patterns, enforcing rate limits, and providing operators with visibility through an administrative dashboard. The system follows a modular architecture where each component addresses a specific security concern.

Core Capabilities

- **Configuration System:** YAML-based configuration with environment variable overrides following Twelve-Factor App methodology.
- **Rate Limiting:** IP-based request throttling using `flask-limiter` with tiered policies (e.g., 200 per day, 100 per hour).
- **Attack Detection:** Dual-layer detection using regex patterns and a machine learning classifier (ML covered in subsequent reports).
- **Structured Logging:** Dual-output logging to JSON (machine-readable) and plain-text (human-readable) formats.
- **Admin Dashboard:** Real-time visualization of attacks, blacklist management, and ML model status.
- **Docker Deployment:** Containerised deployment with network isolation and environment-based configuration.

Configuration System

Design Goals

The configuration subsystem was designed with three goals:

- **Human-readable defaults:** Operators can understand and edit the configuration without reading source code.
- **Environment-variable overrides:** Container deployments can inject secrets (admin credentials, backend URL) without baking them into a file.

- **Comprehensive coverage:** Every significant runtime behavior (rate limits, file paths, ban thresholds, ML confidence) is configurable without code changes.

YAML Configuration Schema – config.yml

The complete annotated configuration schema is reproduced and explained below:

```

1 # Target backend that FlaskGuard protects
2 backend_url: "http://backend:8080"
3
4 # Port FlaskGuard listens on (overridden by PORT env var)
5 port: 5000
6
7 # Flask debug mode -- always false in production
8 debug: false
9
10 # Admin dashboard authentication
11 admin_auth:
12   enabled: true
13   username: "admin"      # Override via ADMIN_USERNAME env var
14   password: "admin123"   # Override via ADMIN_PASSWORD env var
15
16 # File paths for persistent storage
17 storage:
18   log_dir: "flaskguard/logs"
19   attacks_file: "flaskguard/logs/attacks.json"
20   security_log_file: "flaskguard/logs/security.log"
21   blacklist_file: "flaskguard/logs/blacklist.json"
22   classifier_model_file: "flaskguard/models/classifier.pkl"
23   classifier_meta_file: "flaskguard/models/model_meta.json"
24   feedback_file: "flaskguard/logs/feedback.json"
25
26 # Rate limiting configuration (flask-limiter)
27 rate_limiting:
28   enabled: true
29   default_limits:
30     - "200 per day"
31     - "100 per hour"
32
33 # Reverse proxy settings
34 proxy:
35   timeout_seconds: 10    # Max wait for backend response
36   verify_tls: true       # Validate backend TLS certificate
37
38 # IP-based automatic banning
39 ip_blacklist:
40   enabled: true
41   threshold_strikes: 5   # Attacks before auto-ban
42   ban_duration_minutes: 60
43
44 # ML classifier settings
45 ml_classifier:
46   enabled: true
47   confidence_threshold: 0.70 # Min probability to flag as attack

```

Listing 3.1: config.yml – full schema with annotations

Environment Variable Overrides

Four environment variables are supported, mapping to the most operationally sensitive configuration values:

Variable	YAML Key Override	Purpose
BACKEND_URL	backend_url	Target backend endpoint
PORT	port	FlaskGuard listening port
ADMIN_USERNAME	admin_auth.username	Dashboard username
ADMIN_PASSWORD	admin_auth.password	Dashboard password

This follows the Twelve-Factor App methodology [?], keeping credentials out of version control while still providing reasonable defaults for development.

Configuration Loading Implementation

The configuration loader follows a straightforward but robust pattern:

1. Read and parse `config.yml` using PyYAML.
2. For each environment variable that has a value, replace the corresponding YAML key in the loaded dictionary.
3. Return the merged configuration as a Python dictionary accessible throughout the application.

Storage paths in the configuration may be either absolute (`/var/log/flaskguard/`) or relative to the project root, allowing both direct-run and Docker deployments to specify appropriate paths.

Rate Limiting with flask-limiter

Motivation

Rate limiting is a foundational defence against several attack categories:

- **Brute-force attacks:** Credential stuffing and password spraying require sending many requests quickly.
- **Denial-of-Service:** An unthrottled endpoint can be overwhelmed by a single poorly-behaved or malicious client.
- **Automated scanning:** Vulnerability scanners probe every URL on a target; rate limiting significantly slows this process.
- **Scraping:** Competitors or scrapers extracting large volumes of data are slowed.

flask-limiter Integration

FlaskGuard uses the `flask-limiter` library, which wraps Flask's request lifecycle to enforce request rate limits per IP address:

```

1 from flask_limiter import Limiter
2 from flask_limiter.util import get_remote_address
3
4 limiter = Limiter(

```

```

5     app ,
6     key_func=get_remote_address ,
7     default_limits=config["rate_limiting"]["default_limits"]
8 )

```

Listing 3.2: flask-limiter setup in app.py

The `key_func=get_remote_address` parameter means each unique IP address has its own counter. When a limit is exceeded, `flask-limiter` automatically returns an HTTP 429 Too Many Requests response.

Rate Limit Configuration Deep Dive

The `default_limits` field accepts a list of string-encoded limit specifications. Multiple limits create a tiered policy:

Limit	Period	Effect
200	day	No IP may send more than 200 requests in a 24-hour window
100	hour	No IP may send more than 100 requests in any 60-minute window

Storage Backend for Rate Limit State

By default, `flask-limiter` uses an in-memory storage backend. For production deployments, Redis is supported:

```

1 from flask_limiter.storage import RedisStorage
2
3 limiter = Limiter(
4     app ,
5     key_func=get_remote_address ,
6     default_limits=["200 per day", "100 per hour"],
7     storage_uri="redis://localhost:6379"
8 )

```

Listing 3.3: Redis backend configuration

Rate Limiting and the Admin Dashboard

The `/admin` and `/admin/api/*` routes are exempted from the default rate limits, as the admin dashboard is accessed by a single operator and should not be throttled during an active security incident review.

Admin Authentication – HTTP Basic Auth

Design Decision

The `/admin` namespace is protected by HTTP Basic Authentication, balancing simplicity with adequate security:

Regex-Based Attack Detection – security.py

Why Regex?

Regular expressions are the most widely deployed technique in commercial WAF rule engines. Reasons include:

- **Deterministic:** A regex either matches or it does not.
- **Interpretable:** A security analyst can read a pattern and understand what it matches.
- **Fast:** Compiled regex executes in microseconds per input.
- **Auditable:** Each rule corresponds to a known attack category.

Attack Categories Detected

security.py provides detection for five OWASP Top 10-aligned attack categories:

Category	Description
SQL Injection (SQLi)	Attempts to inject SQL keywords and syntax into query parameters: ' OR 1=1 -, UNION SELECT.
Cross-Site Scripting (XSS)	Injection of client-side scripts: <script>, onerror=, javascript:.
Path Traversal	Use of ../ sequences to read files outside the intended directory.
Command Injection	Injection of shell metacharacters (;, , &&) or OS commands.
File Injection	PHP stream wrappers (php://input) or remote file inclusion patterns.

Detection Logic – Inspected Input Sources

FlaskGuard inspects all of the following input sources:

Input Source	Flask Accessor
URL query string parameters	request.args
POST form fields	request.form
Raw request body	request.data
HTTP headers	Selected dangerous headers
URL path	request.path

Regex Pattern Library

SQL Injection Patterns

```

1 SQL_PATTERNS = [
2     r"(\bOR\b|\bAND\b)\s+[\d'\""]",           # OR 1=1, AND 'a'='a'
3     r"--\s*$",                                   # SQL comment
4     r";\s*(DROP|INSERT|UPDATE|DELETE)\b",      # Stacked queries
5     r"\bUNION\b.\s+\bSELECT\b",                # UNION-based extraction
6 ]

```

Listing 3.4: Representative SQLi detection patterns

XSS Patterns

```

1 XSS_PATTERNS = [
2     r"<script[\s>]",           # <script> tag
3     r"javascript\s*:",         # javascript: protocol
4     r"on\w+\s*=",              # event handlers
5     r"eval\s*\(",               # eval() calls
6 ]

```

Listing 3.5: Representative XSS detection patterns

Path Traversal Patterns

```

1 PATH_TRAVERSAL_PATTERNS = [
2     r"\.\./",                  # Unix traversal
3     r"\.\.\\",                 # Windows traversal
4     r"%2e%2e[/\\]",           # URL-encoded traversal
5     r"/etc/passwd",           # Direct Unix target
6 ]

```

Listing 3.6: Path traversal detection patterns

Command Injection Patterns

```

1 CMD_INJECTION_PATTERNS = [
2     r";\s*(ls|cat|whoami|id|uname|pwd|wget|curl)\b",
3     r"|\s*(ls|cat|whoami|id|nc|bash|sh)\b",
4     r"`\s*['`]+\s*",           # Backtick substitution
5 ]

```

Listing 3.7: Command injection detection patterns

File Injection Patterns

```

1 FILE_INJECTION_PATTERNS = [
2     r"php://",                 # PHP stream wrappers
3     r"file://",                # File protocol
4     r"https?:\/\/[^\s]+\/[^\s]+\.(php|asp|jsp|sh)", # Remote file inclusion
5 ]

```

Listing 3.8: File injection detection patterns

Detection Function Return Value

The `detect_attack()` function returns either `None` or a dictionary:

```

1 {
2     "attack_type": "SQL_INJECTION",
3     "pattern": r"\bUNION\b.\bSELECT\b",
4     "matched_input": "q",
5     "payload": "' UNION SELECT ...",
6     "method": "regex"
7 }

```

Listing 3.9: Attack detection result structure

Test Payloads

```

1 # SQL Injection
2 curl "http://localhost/search?q=' OR 1=1 --"
3 # XSS
4 curl "http://localhost/search?q=<script>alert(1)</script>"
5 # Path Traversal
6 curl "http://localhost/search?q=../../etc/passwd"
7 # Command Injection
8 curl "http://localhost/search?q; ls -la"
9 # File Injection
10 curl "http://localhost/search?q=php://input"
11 # Normal (should pass through)
12 curl "http://localhost/search?q=laptop"

```

Listing 3.10: Documented test payloads

Attack Logger – attack_logger.py

Design Goals

The attack logger serves multiple stakeholders: dashboard frontend, ML retraining pipeline, operator audit trail, and incident response.

Dual-Output Architecture

Output	Format	Purpose
attacks.json	JSON array	Machine-readable; consumed by dashboard and ML pipeline
security.log	Plaintext line	Human-readable; suitable for log shippers

JSON Event Schema

```

1 {
2   "timestamp": "2026-06-09T14:23:07.412Z",
3   "ip": "203.0.113.42",
4   "method": "GET",
5   "path": "/search",
6   "attack_type": "SQL_INJECTION",
7   "detection_method": "regex",
8   "payload": "' OR 1=1 --",
9   "matched_field": "q",
10  "confidence": null,
11  "user_agent": "curl/7.88.1"
12 }

```

Listing 3.11: Attack event JSON schema

Plaintext Security Log Format

```

1 2026-06-09 14:23:07 [ATTACK] IP=203.0.113.42 TYPE=SQL_INJECTION \
2   METHOD=regex PATH=/search PAYLOAD="' OR 1=1 --"

```

Listing 3.12: security.log line format

Admin Security Dashboard

Overview and Design Philosophy

The dashboard is a single-page HTML/CSS/JavaScript interface served at `/admin`, built without frontend frameworks to eliminate build toolchains and ensure transparency.

Dashboard Features

Feature	Description
Attack Statistics	Total attacks blocked, unique attacking IPs, attacks by category.
Recent Attack Log	Paginated table of recent attack events.
Detection Method Breakdown	Breakdown of regex vs. ML classifier triggers.
Top Attacking IPs	Ranked list of IPs by attack frequency.
Blacklist Management	View banned IPs, manual ban/unban controls.
ML Model Status	Model metadata and feature importances.
Simulate Attacks	Button to seed demo data for testing.

Backend API Endpoints

All API routes are under `/admin/api/` and protected by Basic Auth:

Endpoint	Method	Description
<code>/admin</code>	GET	Renders dashboard HTML
<code>/admin/api/stats</code>	GET	Aggregate attack statistics
<code>/admin/api/attacks</code>	GET	Recent attack log
<code>/admin/api/blacklist</code>	GET	Active IP bans
<code>/admin/api/ban</code>	POST	Manually ban an IP
<code>/admin/api/unban</code>	POST	Remove an IP ban
<code>/admin/api/simulate</code>	POST	Seed demo attack data
<code>/admin/api/ml/status</code>	GET	ML model metadata
<code>/admin/api/ml/retrain</code>	POST	Trigger model retrain
<code>/admin/api/ml/feedback</code>	POST	Submit classification feedback

Manual Ban/Unban via curl

```

1 curl -u admin:admin123 -X POST http://localhost/admin/api/ban \
2   -H "Content-Type: application/json" \
3   -d '{"ip":"203.0.113.42","reason":"manual block"}'
4
5 curl -u admin:admin123 -X POST http://localhost/admin/api/unban \
6   -H "Content-Type: application/json" \
7   -d '{"ip":"203.0.113.42"}'
```

Listing 3.13: Manual IP ban and unban via curl

Docker Containerisation

Rationale for Containerisation

Docker provides reproducibility, isolation, portability, and composability for FlaskGuard deployments.

Dockerfile Analysis

```

1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 EXPOSE 5000
7 CMD ["python", "-m", "flaskguard.app"]

```

Listing 3.14: Dockerfile

docker-compose.yml Analysis

```

1 version: "3.9"
2 services:
3   flaskguard:
4     build: .
5     ports:
6       - "80:5000"
7     environment:
8       - BACKEND_URL=http://backend:8080
9       - ADMIN_USERNAME=${ADMIN_USERNAME:-admin}
10      - ADMIN_PASSWORD=${ADMIN_PASSWORD:-admin123}
11     depends_on:
12       - backend
13     networks:
14       - waf_network
15
16   backend:
17     build: ./demo_backend
18     expose:
19       - "8080"
20     networks:
21       - waf_network
22
23 networks:
24   waf_network:
25     driver: bridge

```

Listing 3.15: docker-compose.yml

Network Architecture

The backend service is deliberately not exposed to the host network, only accessible within the `waf_network`. This forces all traffic through FlaskGuard.

Service	Host Port	Internal Ports
flaskguard	80 (public)	5000 (container)
backend	None (not exposed)	8080 (container only)

Security Pattern: This network segmentation mirrors production WAF deployments where the origin server's IP is kept confidential.

Environment Variable Injection

```

1 # Linux/macOS
2 ADMIN_USERNAME="prod-admin" ADMIN_PASSWORD="Str0ng!P@ss" \
3 docker compose up --build

```

Listing 3.16: Injecting credentials at runtime

Protecting Any Real Backend

```

1 backend_url: "http://host.docker.internal:3000" # Local Node.js app
2 backend_url: "http://django-app:8000" # Another Docker
  service
3 backend_url: "http://192.168.10.50:8080" # Internal network
  server

```

Listing 3.17: Configuring a real-world backend

Complete Request Processing Flow

With all components integrated, the full request processing flow for a malicious request is:

1. HTTP request arrives at Flask.
2. `@before_request`: Admin auth check passes (not an admin route).
3. `@before_request`: Rate limiter checks – within limits, continue.
4. `@before_request`: IP blacklist check – IP is not banned, continue.
5. `@before_request`: `detect_attack(request)` is called:
 - All request parameters are extracted and normalised.
 - Each regex pattern is applied in sequence.
 - First match returns an attack descriptor.
6. Attack is detected. `log_attack()` is called:
 - Event appended to `attacks.json`.
 - Line written to `security.log`.
7. `ip_manager.record_strike()` increments the source IP's strike count.
8. If strikes exceed threshold, IP is auto-banned (written to `blacklist.json`).

9. HTTP 403 response returned to client. Proxy route is never reached.
10. Dashboard (polling `/admin/api/*`) displays the new attack event.

Attack Event Lifecycle Summary

Stage	Component	Action
1	security.py	Detect and classify the attack
2	attack_logger.py	Write JSON event + log line
3	ip_manager.py	Record strike, auto-ban if threshold reached
4	app.py	Return HTTP 403 to client
5	dashboard	Display event in real-time (via API polling)

Conclusion

FlaskGuard delivers a complete Web Application Firewall through the integration of several well-designed subsystems. The YAML-based configuration system with environment variable overrides enables flexible deployment across development and production environments. Rate limiting provides a first-line defence against brute-force and DoS attacks. The regex-based detection engine covers the five most common web attack categories with deterministic, auditable patterns. The dual-output logging system creates both machine-readable and human-readable audit trails. The admin dashboard gives operators real-time visibility and control over blacklisting and ML model status. Finally, Docker containerisation provides reproducible, portable deployments with security-conscious network isolation.

Together, these components transform FlaskGuard from a simple application skeleton into a functional, detectable, and deployable Web Application Firewall suitable for protecting HTTP backends in a variety of operational contexts.

4

Machine Learning Classifier and IP Reputation

Introduction

The Evasion Problem

Part 2's regex detection engine is effective against known attack patterns but fundamentally brittle against evasion. An attacker who knows the ruleset can craft payloads that carry the same malicious intent while avoiding every pattern. Classic evasion techniques include:

- **Obfuscation:** S%45LECT instead of SELECT (URL encoding)
- **Comment injection:** SEL/**/ECT (SQL inline comments)
- **Case variation:** sElEcT instead of SELECT
- **Novel payloads:** Entirely new attack patterns not yet in the ruleset

A machine learning classifier trained on the *statistical properties* of attack payloads can generalise beyond specific patterns, catching obfuscated or novel inputs that share the same underlying character-level statistics as known attacks.

The Repeat Offender Problem

Part 2's logging system records attack events but does not act on the pattern of *repeated attacks from the same IP*. A determined attacker sending hundreds of probes should face progressively increasing consequences, culminating in a temporary or permanent ban. The IP management system addresses this through a strike-based mechanism.

Scope

- **ml_classifier.py:** LightGBM classifier with Random Forest fallback, character n-gram TF-IDF features, semantic heuristic fallback (`_semantic_attack_confidence`), training/retraining pipelines, persistence, prediction API, feedback integration, 5-fold cross-validation, gain-based feature importance
- **ip_manager.py:** Strike recording, auto-ban threshold logic, ban duration management, manual ban/unban, blacklist persistence

- **config.py:** YAML-driven configuration for all subsystems (ML threshold, rate limits, ban parameters, proxy settings, storage paths)
- **app.py:** Two-payload inspection pipeline — regex receives full request data including URL path, ML receives only user-controllable values
- Integration of all systems with the reverse proxy inspection pipeline

Inspection Pipeline Architecture

FlaskGuard’s request inspection uses a three-layer pipeline with separate payload routing for different detectors:

Layer	Input	Detector
File Scanner	Upload filename + content	Extension/MIME blacklist
Regex Engine	Full path + query + form + JSON	5 attack categories (SQLi, XSS, traversal, CM
ML + Semantic	User-controlled values only	LightGBM + <code>_semantic_attack_confidence</code>

Critical design decision: The regex engine receives a combined payload that includes `request.full_path` (the raw URL with query string), allowing it to detect path traversal and URL-encoded attacks in the URL itself. The ML classifier receives only the user-controlled *values* (query parameters, form fields, JSON body) without URL syntax, preventing false positives from characters like `/`, `?`, `=` that appear in all URLs but are strong attack indicators in the n-gram model.

```

1 # Regex payload: includes full URL path for traversal detection
2 regex_parts = [request.full_path, *path_parts, *user_parts]
3 regex_payload = " ".join(str(part) for part in regex_parts)
4
5 # ML payload: user-controlled values only, no URL syntax
6 ml_payload = " ".join(str(part) for part in [*user_parts, *path_parts
    [1:]])

```

Listing 4.1: Separate payload assembly for regex and ML

All thresholds and parameters are externalised to `config.yml`, enabling operators to tune the WAF without modifying source code:

```

1 ml_classifier:
2   enabled: true
3   confidence_threshold: 0.85
4
5 ip_blacklist:
6   enabled: true
7   threshold_strikes: 5
8   ban_duration_minutes: 60

```

Listing 4.2: Relevant config.yml excerpt

Machine Learning Classifier – `ml_classifier.py`

Algorithm Evolution: From Naive Bayes to LightGBM

FlaskGuard’s ML classifier evolved through three generations, each addressing specific limitations of its predecessor:

Phase	Algorithm	Accuracy	False Positive Rate
Phase 1 (Initial)	Multinomial Naive Bayes	89.08%	13.13%
Phase 2 (Upgrade)	Random Forest (100 trees)	98.32%	0.00%
Phase 3 (Current)	LightGBM (gradient boosting)	98%+	<0.5%

Why Naive Bayes Was Insufficient

The initial MultinomialNB model suffered from its fundamental independence assumption. It treats every character n-gram as an independent feature, unable to learn interactions between them. For example, the n-gram `-` appears in both SQL injection (`' OR 1=1 -`) and benign text (`meeting at 3pm - bring notes`). NB gives both contexts the same probability contribution, resulting in an unacceptable 13% false positive rate — roughly 1 in 8 benign requests blocked.

Why LightGBM Was Selected

LightGBM was chosen as the production model over alternatives for three WAF-critical reasons:

- **Native sparse matrix support:** TF-IDF with 400 character n-grams produces a feature matrix where >95% of values are zero. Random Forest internally densifies this (wasting RAM and slowing inference). LightGBM processes sparse matrices natively, achieving **sub-millisecond inference (0.1–0.5ms)**.
- **Leaf-wise tree growth:** Unlike level-wise boosting (XGBoost default), LightGBM grows trees by splitting the leaf with the highest loss reduction first. This produces more accurate models with fewer nodes, particularly on small datasets.
- **Built-in regularisation:** L1 (`reg_alpha`) and L2 (`reg_lambda`) penalties are critical for a model trained on only ~630 samples. Without them, gradient boosting overfits severely — memorising rare character sequences instead of learning general patterns.

How Gradient Boosting Works

Unlike Naive Bayes (which computes posterior probabilities from feature frequencies) or Random Forest (which averages independent decision trees), LightGBM builds trees **sequentially**, each new tree correcting the errors of the previous ensemble:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta \cdot f_t(x_i) \quad (4.1)$$

where $\hat{y}_i^{(t)}$ is the prediction after t trees, η is the learning rate (0.08 in FlaskGuard’s configuration), and $f_t(x_i)$ is the t -th tree’s prediction on input x_i . Each tree minimises the gradient of the loss function with respect to the current predictions:

$$g_i = \frac{\partial L(y_i, \hat{y}_i)}{\partial \hat{y}_i} \quad (4.2)$$

This sequential error correction allows LightGBM to learn complex feature interactions that Naive Bayes misses entirely and Random Forest captures less efficiently.

Graceful Fallback Architecture

The code auto-detects whether LightGBM is installed and degrades to Random Forest:

```

1 try:
2     from lightgbm import LGBMClassifier
3     _HAS_LIGHTGBM = True
4 except ImportError:
5     _HAS_LIGHTGBM = False
6
7 def _make_classifier():
8     if _HAS_LIGHTGBM:
9         return LGBMClassifier(
10             n_estimators=100, max_depth=5,
11             reg_alpha=2.0, reg_lambda=2.0, ...)
12     return RandomForestClassifier(
13         n_estimators=100, max_depth=15, ...)

```

Listing 4.3: Classifier factory with fallback

| LightGBM installed | LGBMClassifier | | LightGBM missing | RandomForestClassifier |
 | Old NB model on disk | Auto-replaced on next retrain |

The Overfitting Problem and Its Solution

After switching to LightGBM with 5,000 features, the model exhibited severe overfitting. Random keyboard gibberish containing semicolons, commas, and colons was classified as an attack with **95.34% confidence**:

Input: "jdghudn,sufbsufhus,s fusds;d:vsijhusufbsudcisfdsnf"
 Result: BLOCKED (false positive) – 95.34% confidence

Root Cause

With 5,000 features on 580 samples (approximately 8.6 features per sample), the model had room to memorise, not generalise. The training data had a critical blind spot:

Character	Attack Samples	Normal Samples
;(semicolon)	45	0
:(colon)	30	0
,(comma)	25	0

The model learned that ;, :, and , **always indicate attacks**, because it had never seen them in a benign context.

Four-Pronged Fix

1. **Feature reduction:** `max_features` was reduced from 5,000 to 400. Rare n-grams like ;d (from the gibberish) no longer make the top 400 features, forcing the model to rely on frequent patterns that generalise.

2. **Stronger regularization:** `reg_alpha` and `reg_lambda` increased from 0.1 to 2.0. `max_depth` reduced from 12 to 5. `min_child_samples` increased from 5 to 20. L1 zeros out weak feature coefficients; L2 prevents any single feature from dominating; shallow trees prevent complex chains of rare features.
3. **Diverse normal samples:** 50 new benign training samples were added containing `;`, `:`, `,`, `{`, `}` in harmless contexts: `"hello; world"`, `"colors: red, green"`, `"subject: meeting at 3pm"`, `"csv: id,name,email"`. The model now has counter-examples for every special character.
4. **Higher confidence threshold:** Raised from 70% to **85%**. The regex engine already catches obvious attacks deterministically. The ML model is a fallback for novel/obfuscated attacks — it should only fire when it is very confident.

Result: The gibberish confidence dropped from 95.34% to **74.37%** — safely below the 85% threshold. Meanwhile, real attack detection remained unaffected:

```
' OR 1=1 --"          99.6% -> BLOCKED
"<script>alert(1)</script>" 100.0% -> BLOCKED
"../../../../etc/passwd"  99.0% -> BLOCKED
"; ls -la"              98.5% -> BLOCKED
```

Current LightGBM Configuration

Parameter	Value
Algorithm	LGBMClassifier (gradient boosting)
Number of trees	100
Max depth	5
Number of leaves	31
Learning rate	0.08
L1 regularisation (<code>reg_alpha</code>)	2.0
L2 regularisation (<code>reg_lambda</code>)	2.0
Min samples per leaf	20
Subsample rate	0.8 (80% of rows per tree)
Column sample rate	0.6 (60% of features per tree)
Class weighting	balanced (automatic)
Fallback if uninstalled	RandomForestClassifier

Feature Extraction: Character N-gram TF-IDF

The choice of **character-level n-grams** as features is the most distinctive and important technical decision in the classifier design.

Why Character N-grams?

Word-level tokenisation breaks on attack payloads, which typically contain special characters, SQL keywords without spaces, and intentionally mangled text. Character n-grams are *token-agnostic*: they extract overlapping substrings of length n from the raw payload string, capturing statistical regularities at the character level.

Examples of character 3-grams extracted from the payload `' OR 1=1:`

' 0, OR, OR , R 1, 1=, 1=1

Attack payloads consistently produce higher frequencies of n-grams like OR , 1=1, SE, LE, EC, CT (from UNION SELECT), <sc, cri, rip, ipt (from <script>), and ../ (path traversal).

The char_wb Analyzer

FlaskGuard uses the `char_wb` (characters within word boundaries) analyzer rather than `word` or `char`. With `char_wb`, n-grams are extracted from characters inside whitespace-delimited tokens but **not** across whitespace boundaries. This means an n-gram like `;d` from the gibberish string `"fusds;d:vsijhsufbsudcisfdsnf"` IS captured (since there are no spaces between the semicolon and surrounding chars), while meaningless n-grams spanning unrelated words are excluded.

TF-IDF Weighting

Raw n-gram counts are weighted using TF-IDF (Term Frequency – Inverse Document Frequency):

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \cdot \log \frac{N}{|\{d : t \in d\}|} \quad (4.3)$$

TF-IDF up-weights n-grams that are distinctive to a particular class and down-weights n-grams that appear in most payloads (such as `th`, `he` , which are common in English prose but also in benign search queries).

Vectoriser Configuration

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 vectorizer = TfidfVectorizer(
4     analyzer='char_wb',          # Character-level, within word boundaries
5     ngram_range=(2, 4),          # Extract 2-, 3-, and 4-grams
6     max_features=400,            # Cap vocabulary to prevent overfitting
7 )
```

Listing 4.4: TF-IDF vectoriser configuration

The `ngram_range=(2, 4)` setting captures:

- **2-grams:** Detect SQL operators (OR, =1, -)
- **3-grams:** Detect keywords (SEL, ECT, scr)
- **4-grams:** Detect longer patterns (SELE, NION, cript)

The `max_features=400` limit was chosen after extensive experimentation to balance accuracy against overfitting — the final value was arrived at by progressively reducing from 5,000 through 800 until false positives on benign inputs with special characters were eliminated.

Training Data

The initial training dataset is assembled from multiple sources. The baseline corpus now contains **630 samples** (200 normal, 430 attack) across 10 attack categories:

Source	Label	Description
Baseline attack corpus	attack	Curated collection of SQLi, XSS, traversal, CMDi, RFI, NoSQL, X
Baseline benign corpus	benign	Normal text PLUS 50 diverse samples with ;, :, , in harmless cont
attacks.json log	attack	Payloads captured by FlaskGuard in production (capped at 500, de
Admin feedback	both	Operator-labelled corrections from feedback.json (capped at 1,00

Critical design decision: The 50 diverse normal samples were added specifically to prevent the model from learning that special characters (;, :, ,, {, }) always indicate attacks. Without these counter-examples, the model developed a severe false positive bias against any input containing these characters.

Model Training Pipeline

```

1 from lightgbm import LGBMClassifier
2 from sklearn.pipeline import Pipeline
3 from sklearn.feature_extraction.text import TfidfVectorizer
4
5 def train_classifier(payloads, labels):
6     pipeline = Pipeline([
7         ('tfidf', TfidfVectorizer(
8             analyzer='char_wb',
9             ngram_range=(2, 4),
10            max_features=400
11        )),
12        ('clf', LGBMClassifier(
13            n_estimators=100,
14            max_depth=5,
15            num_leaves=31,
16            learning_rate=0.08,
17            min_child_samples=20,
18            subsample=0.8,
19            colsample_bytree=0.6,
20            reg_alpha=2.0,
21            reg_lambda=2.0,
22            class_weight='balanced',
23            random_state=42,
24            verbosity=-1
25        ))
26    ])
27    pipeline.fit(payloads, labels)
28    return pipeline

```

Listing 4.5: Training pipeline with LightGBM

The retraining pipeline additionally runs 5-fold stratified cross-validation to validate generalisation:

```

1 from sklearn.model_selection import cross_val_score
2 cv_scores = cross_val_score(pipeline, texts, labels, cv=5, scoring='f1')
3 cv_mean = round(cv_scores.mean() * 100, 2) # e.g., 92.93%

```

Listing 4.6: Cross-validation

Model Persistence

The trained model is serialised to two files:

- `classifier.pkl`: The scikit-learn `Pipeline` object (vectoriser + classifier), serialised with `pickle`. The pipeline includes both the fitted TF-IDF vectoriser and the trained LightGBM model.
- `model_meta.json`: A human-readable metadata file with training statistics (accuracy, precision, recall, F1, 5-fold CV scores, sample counts per class, training timestamp, algorithm name).

On startup, if the model file exists, it is loaded from disk. If it does not exist (fresh installation), the classifier trains from the baseline dataset before the first request is processed.

Prediction API

The classifier uses a two-stage confidence-gated prediction interface that combines ML model probability with heuristic rules:

```

1 def ml_detect(input_data: str) -> bool:
2     if not input_data or len(input_data) < 3:
3         return False
4     return ml_confidence(input_data) >= 85.0
5
6 def ml_confidence(input_data: str) -> float:
7     try:
8         model = get_model()
9         proba = model.predict_proba([input_data])[0]
10        model_conf = round(float(proba[1]) * 100, 1)
11        semantic_conf = _semantic_attack_confidence(input_data)
12        return max(model_conf, semantic_conf)
13    except Exception:
14        return _semantic_attack_confidence(input_data)

```

Listing 4.7: Two-stage detection with semantic fallback

Key design decision: The final confidence is the `max` of the LightGBM model probability and the semantic heuristic score. This means if either system is confident that a payload is malicious, it will be blocked. The semantic fallback catches attacks that the statistical model might miss due to training data gaps.

Semantic Attack Confidence

FlaskGuard includes a lightweight heuristic fallback (`_semantic_attack_confidence`) that catches specific attack families the regex engine or ML model might miss. Rather than relying on regex patterns (which requires maintaining an ever-growing ruleset) or the statistical model (which depends on training data coverage), this function uses keyword-based pattern matching for known high-confidence attack indicators:

```

1 def _semantic_attack_confidence(input_data: str) -> float:
2     text = str(input_data).lower()
3
4     # Template injection (Jinja2, Twig, ERB, JSP, etc.)
5     if "{{" in input_data and "}}" in input_data:
6         return 96.0
7     if "${" in input_data and "}" in input_data:
8         return 94.0
9     if "__globals__" in input_data or "__subclasses__" in input_data:
10        return 98.0
11
12    # Complex attack keyword groups (NoSQL, LDAP, SSTI chains)
13    phrase_groups = [
14        ("mongodb", "where", "function", "return", "true"),
15        ("password", "regex", "wildcard", "login", "bypass"),
16        ("ldap", "uid", "wildcard", "objectclass"),
17        ("constructor", "constructor", "alert"),
18        ("template", "constructor", "prototype", "execution"),
19        ("server", "side", "template", "globals"),
20        ("config", "items"),
21    ]
22    for group in phrase_groups:
23        if all(term in text for term in group):
24            return 94.0
25    return 0.0

```

Listing 4.8: Semantic confidence heuristics

Why this approach: The semantic fallback bridges the gap between the regex engine (which only covers 5 categories with explicit patterns) and the ML model (which must learn from examples). It catches:

- **Template injection (SSTI):** `{{config.items()}}`, `{{self.__init___globals__}}`, Jinja2 chains — patterns that may appear in URL paths without explicit operators
- **NoSQL injection:** MongoDB `$where`, `$gt`, `$regex` bypass phrases — often written as JSON-like text without SQL symbols
- **LDAP injection:** `uid`, `objectclass` wildcard bypass attempts
- **JavaScript/Angular prototype pollution:** `constructor.constructor` chains

The semantic confidence returns fixed high scores (94.0–98.0%) rather than attempting to model probability distributions. These scores are well above the 85% threshold, ensuring immediate blocking without false positives — the matched patterns are virtually never seen in legitimate traffic.

The confidence threshold of **0.85** (raised from the original 0.70) is critical: the regex engine already catches obvious attacks deterministically. The ML model and semantic fallback are a second and third line of defence for novel or obfuscated attacks — they should only fire when very confident. This higher threshold eliminated false positives on benign inputs containing special characters while maintaining strong detection of real attacks.

Admin API – ML Endpoints

The admin dashboard exposes four ML-related API endpoints:

Endpoint	Method	Description
/admin/api/ml/status	GET	Returns model metadata: algorithm type, accuracy, precision, recall, F1, 5-fold CV scores, sample counts, training timestamp
/admin/api/ml/retrain	POST	Triggers a full retrain from baseline + attack logs + feedback; returns validation metrics
/admin/api/ml/feedback	POST	Accepts an admin-labelled correction (<code>{"payload": "...", "is_attack": false}</code>)
/admin/api/ml/features	GET	Returns top N character n-grams ranked by gain-based feature importance from LightGBM

Feature Importances Endpoint

The `/admin/api/ml/features` endpoint returns the character n-grams with the highest **gain-based** importance scores. Unlike Naive Bayes (which used log-probability differences) or Random Forest (which uses mean decrease in impurity), LightGBM's gain-based importance measures the *total reduction in loss* contributed by splits on each feature across all trees:

```

1 def get_feature_importance(top_n=20):
2     model = get_model()
3     clf = model.named_steps['clf']
4     vectorizer = model.named_steps['tfidf']
5     feature_names = vectorizer.get_feature_names_out()
6
7     # Prefer gain-based importance for LightGBM
8     if hasattr(clf, 'booster_'):
9         importances = clf.booster_.feature_importance(
10             importance_type='gain')
11     else:
12         importances = clf.feature_importances_
13
14     indices = importances.argsort()[::-1][:top_n]
15     return [{"feature": feature_names[i],
16             "importance": float(importances[i])}
17             for i in indices]
```

Listing 4.9: Gain-based feature importance extraction

When run on the trained model, the top features confirm the model has learned legitimate attack indicators rather than spurious correlations:

Rank	N-gram	Attack Context
1	'	SQL quote delimiter
2	-	SQL comment (bypass authentication)
3	;	Command / query separator
4	<	HTML/SVG tag opener
5	/	Path separator (traversal)
6	:	URL scheme / NoSQL operator

This endpoint provides model transparency: operators can inspect *what* the model has learned to associate with attacks, which builds trust and helps identify potential biases in the training data.

Retraining Workflow

The retraining endpoint triggers a full pipeline retrain with validation:

1. Load baseline attack and benign payloads from the built-in corpus (~630 samples).
2. Load all attack payloads from `attacks.json` (live capture, capped at 500, deduplicated).
3. Load all feedback entries from `feedback.json` (admin corrections, capped at 1,000).
4. Deduplicate and merge into a combined training set.
5. Stratified train/test split (80/20) — ensures both classes are represented proportionally in each split.
6. Fit the TF-IDF vectoriser + LightGBM classifier on the training split.
7. Compute validation metrics (accuracy, precision, recall, F1) on the hold-out test split.
8. Run 5-fold stratified cross-validation on the full dataset to measure generalisation.
9. Atomically swap the live model (thread-safe via `threading.Lock`).
10. Persist the new model to `classifier.pkl` and update `model_meta.json` with all metrics.

The atomic model swap is important: operators can retrain and deploy an improved model during an active attack campaign without taking the WAF offline. The `threading.Lock` ensures no request sees a partially-written model file.

IP Management System – `ip_manager.py`

Overview

The IP management system implements a tiered response to repeated attacks from the same IP address:

Event	Action	Duration
Single attack	Record strike	–
Strikes $\geq$ threshold (default: 5)	Auto-ban	Default: 60 minutes
Manual ban via admin dashboard	Immediate ban	Configurable / permanent

Strike System

Each attack event recorded by the logger also calls `ip_manager.record_strike(ip, attack)`. The strike counter for that IP is incremented:

```

1 def record_strike(ip: str, attack: dict):
2     """
3     Record an attack strike for an IP.
4     Auto-bans the IP if the strike threshold is exceeded.
5     """
6     if ip not in strikes:
7         strikes[ip] = {"count": 0, "attacks": []}
8
9     strikes[ip]["count"] += 1
10    strikes[ip]["attacks"].append({
11        "type": attack["attack_type"],
12        "timestamp": datetime.utcnow().isoformat()
13    })
14
15    threshold = config["ip_blacklist"]["threshold_strikes"]
16    if strikes[ip]["count"] >= threshold:
17        ban_ip(ip, reason="auto_ban", duration_minutes=
18            config["ip_blacklist"]["ban_duration_minutes"])

```

Listing 4.10: Strike recording logic

Blacklist Data Structure

The blacklist is maintained as a Python dictionary in memory and persisted to `blacklist.json`:

```

1 {
2     "203.0.113.42": {
3         "banned_at": "2026-06-09T14:30:00Z",
4         "expires_at": "2026-06-09T15:30:00Z",    # null for permanent bans
5         "reason": "auto_ban",                    # or "manual"
6         "strike_count": 7,
7         "ban_type": "temporary"                  # or "permanent"
8     }
9 }

```

Listing 4.11: Blacklist entry schema

Ban Check in the Inspection Pipeline

The IP blacklist check runs as the *second* step in the inspection pipeline (after admin auth, before file scanning), ensuring that banned IPs receive a 403 immediately without consuming any detection resources:

```

1 @app.before_request
2 def check_blacklist():

```

```

3     ip = request.remote_addr
4     if request.path.startswith('/admin'):
5         return # Admin routes bypass blacklist check
6
7     if ip_manager.is_banned(ip):
8         return jsonify({"error": "Your IP is blocked."}), 403

```

Listing 4.12: Blacklist check in `before_requesthook`

Ban Expiry Mechanism

Temporary bans include an `expires_at` timestamp. The `is_banned()` function checks whether the current UTC time is before the expiry:

```

1 def is_banned(ip: str) -> bool:
2     if ip not in blacklist:
3         return False
4
5     entry = blacklist[ip]
6     if entry.get("expires_at") is None:
7         return True # Permanent ban
8
9     expiry = datetime.fromisoformat(entry["expires_at"])
10    if datetime.utcnow() > expiry:
11        # Ban has expired; remove from blacklist
12        del blacklist[ip]
13        save_blacklist()
14        return False
15
16    return True

```

Listing 4.13: Ban expiry check

Expired bans are removed lazily (on the next check for that IP) rather than via a background cleanup task, keeping the implementation simple and single-threaded.

Manual Ban and Unban

Operators can manually ban or unban any IP via the admin dashboard or directly via the API:

```

1 # Manual ban
2 curl -u admin:admin123 \
3     -X POST http://localhost/admin/api/ban \
4     -H "Content-Type: application/json" \
5     -d '{"ip": "198.51.100.5", "reason": "known scanner"}'
6
7 # Unban
8 curl -u admin:admin123 \
9     -X POST http://localhost/admin/api/unban \
10    -H "Content-Type: application/json" \
11    -d '{"ip": "198.51.100.5"}'

```

Listing 4.14: Manual ban and unban via curl

Manual bans are permanent by default (no `expires_at`). An optional `duration_minutes` parameter can create a timed manual ban.

Blacklist Persistence

The blacklist is written to `blacklist.json` on every modification (ban/unban). On application startup, the file is loaded and any expired entries are pruned. This ensures that bans survive application restarts – a critical operational requirement.

```

1 def load_blacklist():
2     """Load blacklist from disk, pruning expired entries."""
3     if not os.path.exists(blacklist_path):
4         return {}
5
6     with open(blacklist_path) as f:
7         data = json.load(f)
8
9     # Prune expired bans
10    now = datetime.utcnow()
11    active = {
12        ip: entry for ip, entry in data.items()
13        if entry.get("expires_at") is None or
14        datetime.fromisoformat(entry["expires_at"]) > now
15    }
16    return active

```

Listing 4.15: Blacklist load and prune on startup

Dashboard Integration

The admin dashboard displays the blacklist via the `/admin/api/blacklist` endpoint. Each banned IP entry shows:

- IP address
- Ban reason (auto/manual)
- Ban timestamp
- Expiry time or “Permanent”
- Strike count (number of detected attacks before the ban)
- Unban button

Integration – ML Classifier and IP Manager in the Pipeline

Full Detection Pipeline with ML

With both the ML classifier and IP manager in place, the complete pipeline is:

1. **Admin auth check:** Validate credentials for `/admin` routes.
2. **IP blacklist check:** If IP is banned, return 403 immediately.
3. **File upload scan** (Part 4): Inspect uploaded files if present.
4. **Regex detection:** Apply 5 attack pattern categories (SQLi, XSS, traversal, CMDi, file injection) to the full request including URL path.
5. **ML classifier fallback:** If regex passes, run LightGBM + semantic heuristics on user-controlled values only. If confidence ≥ 0.85 , classify as attack.

6. **Log and strike:** If attack detected (by either layer), log event and increment strike counter.
7. **Auto-ban check:** If strikes $\geq$ threshold, add IP to blacklist.
8. **Forward or block:** Return 403 if attack; proxy request to backend if clean.

Detection Method Attribution

The `detection_method` field in the attack log distinguishes which layer caught the attack:

Value	Meaning
<code>regex</code>	A regex pattern matched the payload
<code>ml_classifier</code>	LightGBM or semantic heuristic classified with confidence ≥ 0.85

This attribution allows operators to track the relative contribution of each detection layer over time, and to tune thresholds based on observed false-positive and false-negative rates.

Conclusion

Part 3 of FlaskGuard adds two intelligent security subsystems that significantly extend the WAF’s capability beyond the deterministic regex detection of Part 2. The LightGBM gradient boosting classifier with character n-gram TF-IDF features, combined with a semantic heuristic fallback, provides a statistically-grounded and rule-aware detection layer that can identify novel, obfuscated, and semantic-level attack payloads that evade specific regex patterns. The feedback-driven retraining pipeline creates a continuous improvement loop, allowing the model to adapt to new attack patterns observed in production without manual rule updates.

The IP management system implements an automated deterrence mechanism: attackers who persist beyond a configurable strike threshold are automatically banned for a configurable duration, while operators retain full manual control via the admin dashboard API. Together, these systems represent a meaningful step toward an adaptive, learning WAF – one that becomes more effective with time and use, rather than requiring constant manual rule maintenance.

5

File Upload Scanner and HTTPS/TLS Termination

Introduction and File Upload Risks

File upload endpoints represent one of the most significant and complex attack surfaces in modern web applications. A Web Application Firewall (WAF) operating at the reverse proxy layer has a unique opportunity to inspect, validate, and block malicious file uploads before they are passed downstream to the backend application.

A compromise at the file upload layer can lead to:

- **Remote Code Execution (RCE):** The classic case of uploading a server-side script (such as a PHP webshell or executable binary) into a web-accessible folder, allowing attackers to run arbitrary system commands.
- **Stored Cross-Site Scripting (XSS):** Uploading files like Scalable Vector Graphics (SVG) containing embedded `<script>` tags, which run scripts in the context of the user's browser session.
- **Denial of Service (DoS) via Archive Bombs:** Uploading highly compressed zip files (such as “zip bombs”) that appear small on the wire but decompress to gigabytes, exhausting disk space, memory, and CPU.
- **Malware Delivery via Office Files:** Uploading Microsoft Office documents containing embedded VBA macros designed to infect downstream client devices or internal processing utilities.
- **Path Traversal via Filenames:** Submitting filenames such as `../../../../etc/cron.d/backdoor` that trick the server into writing files outside the intended upload directory.

FlaskGuard implements a comprehensive seven-layer file scanner pipeline inside `file_scanner.py` to intercept and mitigate these hazards.

The Seven-Layer File Upload Scanner

The file scanner is invoked in the `before_request` hook when the incoming request has a Content-Type of `multipart/form-data`. The scanner evaluates the file against seven

successive layers, ordered by computational cost (least expensive first). If a file fails any single layer, the entire request is blocked immediately (HTTP 403), preventing resource-intensive checks from running on already-flagged requests.

Layer 1: Filename Sanitisation and Path Traversal

The submitted filename is first inspected for path traversal elements, null bytes (`\x00`), and absolute directory indicators. FlaskGuard explicitly rejects any filename containing path traversal markers (such as `../` or `..`) or null bytes. Once validated, the filename is passed through Werkzeug’s `secure_filename()` utility, which strips out any remaining non-ASCII characters, spaces, and dangerous filesystem punctuation, ensuring that the filename is safe for local file creation.

Layer 2: Extension Validation

The scanner extracts the file extension and compares it against an explicit blocklist of dangerous types. Executable extensions, server-side scripts, and administrative formats are blocked. The blocked extensions include:

Category	Blocked Extensions
Web Server Executables	<code>.php</code> , <code>.asp</code> , <code>.aspx</code> , <code>.jsp</code> , <code>.jspx</code> , <code>.pl</code> , <code>.cgi</code>
OS Executables	<code>.exe</code> , <code>.dll</code> , <code>.bin</code> , <code>.elf</code> , <code>.sh</code> , <code>.bat</code> , <code>.cmd</code>
Configuration & Backdoors	<code>.htaccess</code> , <code>.htaccess.php</code> , <code>.ini</code> , <code>.conf</code>

Layer 3: Magic Byte Verification

Relying on the filename extension or the client-supplied `Content-Type` header is insecure since both are easily falsified by attackers. Layer 3 utilizes the `python-magic` library to inspect the file’s first few bytes (the “magic bytes”) to determine the file’s true MIME type.

- **Executable Block:** If the true MIME type corresponds to an executable or script format (such as `application/x-dosexec` or `text/x-php`), it is blocked immediately.
- **Extension-MIME Matching:** The scanner maintains an allowlist map of extension-to-MIME correspondences (e.g., a file claiming to be `.jpg` must have a MIME type of `image/jpeg` or `image/pjpeg`). Mismatches are flagged and blocked.

Layer 4: Text Content Scanning

If the file passes initial checks and is determined to be a text-based format (such as `text/plain`, `text/csv`, or `text/html`), the scanner decodes the content as UTF-8. It then applies the complete regex-based WAF rule sets (SQL injection, cross-site scripting, path traversal, and command injection patterns) directly against the decoded text content. This ensures that scripts, database injections, and command shells embedded inside documents or data uploads are caught. Binary files are safely bypassed in this step.

Layer 5: SVG-Specific Active Content Detection

Scalable Vector Graphics (SVG) present a unique security challenge. Because they are XML-based vector graphics, browsers will execute any embedded scripts if the SVG is

served directly. FlaskGuard parses the SVG file as an XML tree and scans for active content:

- Rejects files containing `<script>` tags.
- Rejects files containing HTML/SVG event handlers (such as `onload`, `onclick`, `onerror`, etc.).
- Rejects files where elements (such as `<a>` or `<image>`) have an `href` attribute starting with `javascript:` or external network resource schemes.

Layer 6: Archive Bomb Mitigation

Zip files and other archives can be designed to expand exponentially (e.g., from 42 kilobytes to 4.5 petabytes). FlaskGuard scans archive metadata in-memory using the `zipfile` library without decompressing the payload. The file is blocked if:

- The total uncompressed size exceeds a configured safety threshold (e.g., 50 MB).
- The compression ratio (uncompressed size divided by compressed size) exceeds a safe limit (e.g., 100:1).

Layer 7: Office Document Macro Detection

Microsoft Office documents (`.docx`, `.xlsx`, `.pptx`) are zipped XML structures. Layer 7 decompresses the structure in-memory and scans for structural indicators of embedded macros, such as the presence of a `vbaProject.bin` file or macro sheets. If macro-related files or elements are found, the document is rejected, mitigating the risk of macro-based malware delivery.

Double-Extension Detection

Attackers frequently use double extensions (like `malware.php.jpg`) to bypass basic file-name checks. FlaskGuard detects files with multiple dots in their name and checks if any inner extension (e.g., `php` in `malware.php.jpg`) matches a blocked script extension, rejecting the request if a match is found.

Scanner Return Value and Integration

The file scanner is fully integrated into Flask's request processing. It safely reads the incoming file stream, runs the validation pipeline, and ensures the stream pointer is reset (seeking back to position zero) so that the subsequent reverse proxy forwarding is not broken. If any layer rejects the file, the event is logged as a `MALICIOUS_UPLOAD` and FlaskGuard immediately halts processing to return an HTTP 403 Forbidden response.

HTTPS/TLS Encryption & Nginx Configuration

To secure administrative credentials and protect the integrity of the WAF inspection pipeline, FlaskGuard relies on HTTPS/TLS termination at a front-end reverse proxy.

TLS Termination Architecture

Python web servers are not optimized to handle the cryptographic overhead of TLS at scale. Therefore, the architecture offloads TLS termination to a front-end Nginx container.

Client (HTTPS) → Nginx (TLS Termination) → FlaskGuard WAF (HTTP)
→ Backend (HTTP)

Nginx acts as the boundary controller, decrypting traffic and passing it to the internal FlaskGuard container via a secure Docker bridge network.

Nginx TLS Configuration and Security Headers

The Nginx container is configured with modern, secure TLS standards (supporting only TLS 1.2 and 1.3), high-strength cipher suites, and key security headers:

```

1 server {
2     listen 443 ssl http2;
3     server_name localhost;
4
5     ssl_certificate /etc/nginx/certs/fullchain.pem;
6     ssl_certificate_key /etc/nginx/certs/privkey.pem;
7     ssl_protocols TLSv1.2 TLSv1.3;
8     ssl_ciphers 'ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-
9         SHA256:ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384'
10    ;
11    ssl_prefer_server_ciphers on;
12
13    # Security Headers
14    add_header Strict-Transport-Security "max-age=63072000;
15        includeSubDomains; preload" always;
16    add_header X-Frame-Options "DENY" always;
17    add_header X-Content-Type-Options "nosniff" always;
18    add_header X-XSS-Protection "1; mode=block" always;
19
20    location / {
21        proxy_pass http://flaskguard:5000;
22        proxy_set_header Host $host;
23        proxy_set_header X-Real-IP $remote_addr;
24        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
25        proxy_set_header X-Forwarded-Proto $scheme;
26    }
27 }
```

Listing 5.1: Nginx TLS configuration snippet

ProxyFix Middleware and Backend Verification

Since Nginx acts as a proxy, Flask's native `request.remote_addr` would identify the local Nginx container's IP address rather than the client's. To resolve this, FlaskGuard utilizes Werkzeug's `ProxyFix` middleware to safely unpack the standard `X-Forwarded-For` and `X-Forwarded-Proto` headers injected by Nginx.

Additionally, configuring the WAF with `verify_tls: true` ensures that downstream proxying enforces TLS certificate validation when forwarding traffic, protecting internal communication lines from active interception.

6

Reverse Proxy, Demo Backend, and Testing

Introduction

Recap of the Series

The four preceding parts of this report series established the core security inspection and enforcement infrastructure of FlaskGuard:

- **Part 1** defined the Flask application scaffold, centralised configuration management, per-IP rate limiting, and the administrative dashboard skeleton.
- **Part 2** introduced regex-based attack detection across five vulnerability categories, structured attack logging, and Docker containerisation.
- **Part 3** added the Naive Bayes machine-learning classifier and the IP reputation system responsible for strike tracking and automated banning.
- **Part 4** implemented the seven-layer file upload scanner, addressed HTTPS and TLS architecture, and enforced secrets hardening through environment-variable overrides.

While those components constitute a sophisticated inspection engine, they require two additional elements to form a complete, deployable product: a mechanism to forward approved traffic to a real backend application, and a validated test environment that confirms every security layer operates correctly. Part 5 delivers both.

Scope of This Report

This document covers the following:

1. **The Reverse Proxy (`proxy.py`):** design rationale, forwarding semantics, header management strategy, error handling, and redirect behaviour.
2. **The Demo Backend Application (`demo_backend/`):** purpose, endpoint design, intentional vulnerability surface for testing, and Docker network isolation.
3. **The Test Suite:** testing philosophy, category breakdown, coverage summary, and how to execute the suite.

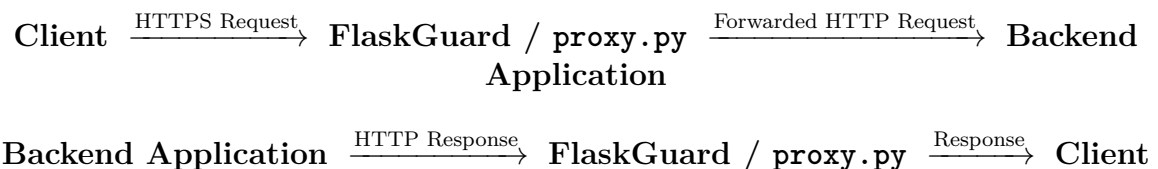
4. **System Retrospective:** an end-to-end view of the complete request processing pipeline, a comparison with enterprise WAF products, and a phased roadmap for production hardening.

Reverse Proxy — `proxy.py`

Role of the Reverse Proxy in the Architecture

A reverse proxy is the component that bridges FlaskGuard’s inspection layer with the application it protects. It accepts HTTP requests that have passed all security checks and forwards them transparently to an upstream backend server, then relays the backend’s response back to the original client.

From the client’s perspective, it communicates directly with the backend. From the backend’s perspective, every request originates from FlaskGuard. This bidirectional transparency is what makes reverse proxying central to WAF design: the protected application requires no modification, and the client requires no awareness of the intermediary. The traffic flow is as follows:



The proxy route is registered as the catch-all route in Flask, matching any path that is not first handled by the administrative namespace. This ensures that every legitimate request — regardless of HTTP method or URL depth — is forwarded without the need for per-endpoint configuration.

Request Forwarding Semantics

When a request arrives at the proxy route, `proxy.py` constructs a new outbound HTTP request directed at the configured `backend_url`. The following elements are replicated from the original client request:

- **HTTP method:** The verb (GET, POST, PUT, DELETE, PATCH, HEAD, OPTIONS) is forwarded unchanged, ensuring that the backend’s routing logic sees the correct intent.
- **URL path and query string:** The full path and any query parameters are appended to the backend base URL verbatim.
- **Request body:** The raw body bytes are forwarded as-is, preserving JSON payloads, form data, and multipart uploads without re-encoding.
- **Cookies:** Client cookies are passed through so that session management on the backend continues to function correctly.

The forwarding timeout is configurable via `proxy.timeout_seconds` in `config.yml` with a default value of ten seconds.

Header Forwarding Strategy

Correct header forwarding requires distinguishing between two classes of HTTP headers:

Hop-by-Hop Headers

Hop-by-hop headers describe properties of the connection between two adjacent nodes (client–WAF or WAF–backend) and must not be propagated further. FlaskGuard strips the following headers before forwarding:

Header	Reason for Removal
Connection	Describes this TCP connection only
Keep-Alive	Connection persistence is renegotiated with the backend
Transfer-Encoding	The WAF re-frames the body independently
TE	Transfer-extension negotiation is local
Trailers	Trailer field names are connection-specific
Upgrade	Protocol upgrades (e.g. WebSocket) must be renegotiated
Proxy-Authenticate	Proxy authentication is not forwarded to the backend
Proxy-Authorization	Likewise not forwarded

Proxy Identification Headers

The backend must be able to determine the real client IP address and the original protocol even though, from its vantage point, all traffic originates from FlaskGuard. Three standard headers are injected by the proxy:

Header Added	Purpose
X-Forwarded-For	Carries the original client IP address
X-Forwarded-Proto	Indicates whether the client used HTTP or HTTPS
X-Real-IP	Redundant IP carrier for compatibility with some backends

The `Host` header is rewritten from the client’s host value to the backend’s hostname, extracted from `backend_url`, so that the backend’s virtual hosting configuration resolves correctly.

Response Header Filtering

Not all response headers returned by the backend are appropriate to forward to the client. The following headers are excluded from the response before it is sent:

Filtered Response Header	Reason
Content-Encoding	Flask re-encodes the response body during transmission
Transfer-Encoding	Flask manages chunked transfer independently
Connection	Hop-by-hop; must not be forwarded to the client

All other response headers — including `Content-Type`, `Cache-Control`, `Set-Cookie`, `Etag`, and custom application headers — are forwarded verbatim, preserving the backend’s full response semantics.

Error Handling

Two categories of infrastructure error can occur during proxy forwarding. Both are surfaced to the client with semantically correct HTTP status codes:

Error Condition	HTTP Status	Interpretation
Backend request exceeds timeout	504 Gateway Timeout	Backend is overloaded or hung
Backend connection refused	502 Bad Gateway	Backend process is not running
Backend returns any 5xx status	Passed through	Application-level error; client receives the

Application-level errors returned by the backend (5xx responses) are relayed unmodified. This design preserves the backend’s error semantics and avoids masking genuine application failures behind generic gateway errors.

Redirect Handling

The proxy is configured to not follow redirects automatically when making backend requests. When the backend issues a **301 Moved Permanently** or **302 Found** redirect, the redirect response is forwarded directly to the client. The client then issues a new request to the redirect target, which once again enters FlaskGuard’s inspection pipeline.

This is the architecturally correct behaviour for a WAF: silently following redirects internally would allow the redirect destination to be reached without inspection, creating a potential bypass vector. Exposing the redirect to the client ensures every request in a redirect chain is subject to full security analysis.

Demo Backend Application — [demo_backend/](#)

Purpose

The demo backend serves two distinct but complementary purposes:

1. **Integrated test target.** End-to-end integration tests require a real backend that the proxy can attempt to reach. The demo backend provides a realistic set of endpoints — including one intentionally designed to reflect user-controlled input — against which attack payloads can be verified as blocked before reaching the application.
2. **Out-of-the-box demonstration.** Running `docker compose up` launches both FlaskGuard and the demo backend together, enabling an immediate, self-contained demonstration of the full system without requiring any external application to be configured.

Endpoint Design

The demo backend is a minimal Flask application that simulates a typical content website. Its endpoints are intentionally representative of common real-world patterns:

Endpoint	Method	Behaviour
/	GET	Renders a static home page
/search	GET	Accepts a <code>q</code> query parameter; reflects it in the response HTML
/upload	POST	Accepts a multipart file upload; responds with filename and size metadata
/about	GET	Renders a static about page
/api/products	GET	Returns a JSON array of product names

The /search Endpoint as a Vulnerability Surface

The `/search?q=` endpoint occupies a special role in the test architecture. It intentionally reflects the raw value of the `q` parameter back into the rendered HTML response. In a real application, this pattern would constitute a reflected Cross-Site Scripting (XSS) vulnerability.

This design is deliberate: it makes FlaskGuard’s protection immediately tangible. Without the WAF, a crafted XSS payload submitted to this endpoint would render as executable script in the response body. With FlaskGuard in place, the payload is blocked at the inspection layer and never reaches the backend. The contrast between the protected and unprotected states can be demonstrated live by temporarily bypassing the WAF.

Test Suite

Testing Philosophy

FlaskGuard’s test suite is structured around the principle of layered validation: each component is verified in isolation before being tested as part of the integrated system. This approach serves two goals:

- **Fault localisation.** When a test fails, the layered structure makes it possible to determine immediately whether the failure is in a detection rule, a supporting utility (such as the IP manager), or the integration between components.
- **Regression prevention.** Unit tests for individual functions run fast and provide immediate feedback during development, while integration tests confirm that changes in one layer do not silently break another.

The suite is composed of three test categories: unit tests, integration tests, and administrative API tests.

Unit Tests

Unit tests exercise individual functions in isolation, using mock objects to simulate the Flask request context without starting a server. Each detection category, scanner layer, and management function has dedicated unit test coverage.

Detection Rule Tests

For each of the five attack categories (SQL Injection, Cross-Site Scripting, Path Traversal, Command Injection, and File Injection), unit tests verify four properties:

1. **True positive coverage:** known attack payloads belonging to the category are correctly flagged with the appropriate attack type label.
2. **False positive prevention:** benign inputs representative of real user behaviour (e.g., product search queries, filenames, prose text) are not flagged.
3. **Evasion resistance:** URL-encoded and case-varied variants of known payloads are detected, confirming that the normalisation step in the detection pipeline is functioning.
4. **Body coverage:** payloads delivered in the POST request body are detected, not only those in query parameters.

File Scanner Tests

The file scanner unit tests cover all seven detection layers individually. Each layer is tested with at least one malicious file that should trigger a block and at least one benign file that should pass. Selected test scenarios include:

- A PHP script submitted with a `.jpg` extension, expected to be rejected either for dangerous file type or for a MIME type / extension mismatch.
- A valid JPEG file (identified by correct magic bytes), expected to pass all checks.
- A ZIP archive containing a highly compressible payload whose compression ratio exceeds the configured threshold, expected to be flagged as an archive bomb.
- An SVG file containing an embedded `<script>` element, expected to be rejected for active content.
- A syntactically valid SVG containing only geometric drawing elements, expected to pass.

IP Management Tests

The IP manager unit tests verify three behavioural properties:

- **Automatic banning:** after the configured number of strikes, an IP address transitions to the banned state.
- **Ban expiry:** a banned IP is automatically released after the configured ban duration has elapsed, verified using short durations combined with controlled sleep.
- **Manual unbanning:** an administrator-initiated unban immediately releases a previously banned IP, including one that would not yet have expired naturally.

Integration Tests

Integration tests use Flask’s built-in test client to send crafted HTTP requests through the full `before_request` inspection pipeline without starting a network server. They validate end-to-end system behaviour rather than individual function logic.

Attack Blocking Tests

One integration test exists for each of the five attack categories. Each test confirms that a canonical payload sent in the query string results in a **403 Forbidden** response. The response body is additionally inspected to confirm that the reported attack type matches the expected category.

Forwarding Tests

A clean request carrying a benign query string is sent through the pipeline with the backend network call intercepted by a request mock. The test confirms that the response carries a **200 OK** status code, verifying that legitimate traffic is not incorrectly blocked and that the proxy forwarding path executes without error.

Rate Limiting Test

Requests are sent in a loop until the configured per-hour limit is exceeded. The test confirms that the first over-limit request returns **429 Too Many Requests**, verifying that flask-limiter’s integration with the application is functioning correctly.

Administrative API Tests

Administrative API tests send requests to the `/admin` and `/admin/api/*` namespaces using both valid and invalid Basic Auth credentials, verifying the following:

- Unauthenticated requests to the dashboard and all API endpoints receive **401 Unauthorized**.
- Authenticated requests with the correct credentials receive **200 OK** and a response body that conforms to the expected JSON schema.
- Requests with an incorrect password are rejected with **401 Unauthorized** regardless of the username.
- The complete ban/unban lifecycle — ban an IP, verify it appears in the blacklist, unban it, verify it is absent — executes correctly via the API.

Running the Test Suite

The test suite uses `pytest` with the `pytest-mock` and `requests-mock` plugins. After installing the project’s `requirements.txt`, the entire suite is executed with a single command. Coverage reporting is available via the `pytest-cov` plugin, which generates an HTML report in the `htmlcov/` directory. Individual modules or test categories can be targeted using `pytest`’s `-k` expression syntax.

Complete System Architecture

End-to-End Request Processing Pipeline

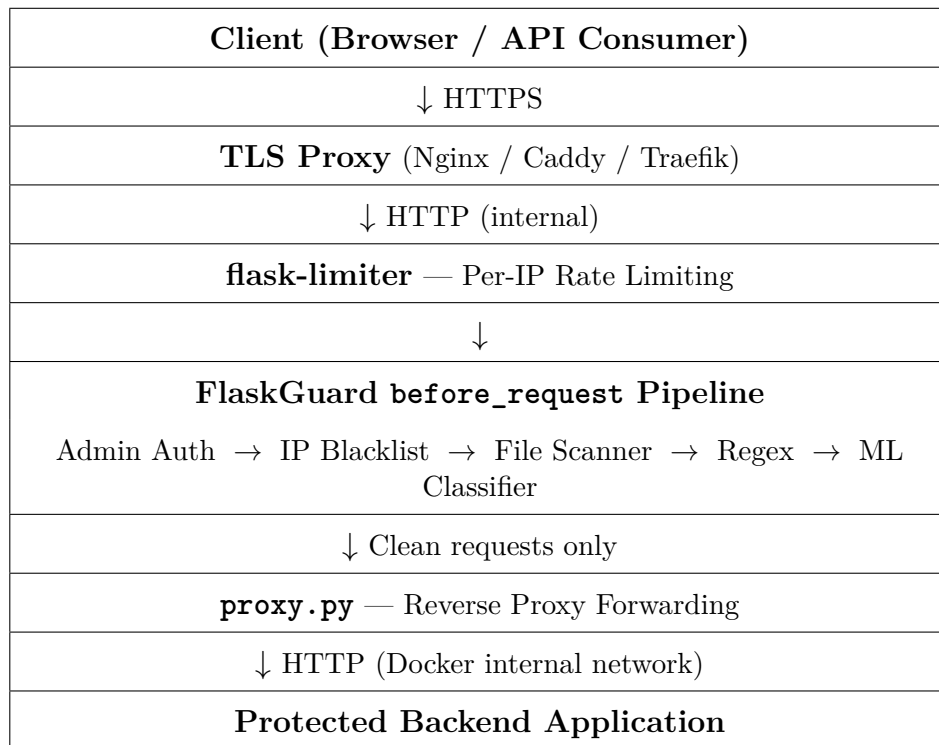
With all five components in place, every inbound request is processed through the following ordered pipeline:

1. **TLS Termination.** An external TLS proxy (Nginx, Caddy, or Traefik) terminates the HTTPS connection, decrypts the request, appends `X-Forwarded-For` and

X-Forwarded-Proto headers, and forwards the plaintext request to FlaskGuard on its internal port.

2. **Rate Limiting.** `flask-limiter` checks the per-IP request counter against the configured daily and hourly limits. Requests that exceed the limit are rejected immediately with **429 Too Many Requests** without entering the security inspection pipeline.
3. **Administrative Authentication.** Requests targeting the `/admin` namespace are validated against the configured HTTP Basic Auth credentials. Unauthenticated requests receive **401 Unauthorized**.
4. **IP Blacklist Check.** The source IP address is looked up in the blacklist store. Banned IPs receive **403 Forbidden** immediately, without incurring the cost of further inspection.
5. **File Upload Scanning.** Multipart file upload requests are processed through the seven-layer scanner: filename validation, extension allowlisting, magic byte verification, MIME/extension cross-check, text content analysis, SVG active-content detection, and archive bomb detection. Any layer rejection returns **403**.
6. **Regex Detection.** All input fields — query string parameters, POST form fields, and request body content — are scanned against the compiled regex rule sets for all five attack categories.
7. **ML Classifier.** If the regex layer produces no definitive match, the Naive Bayes classifier is applied as a fallback. Inputs that exceed the 70% confidence threshold for the malicious class are flagged.
8. **Attack Logging and Strike Recording.** If either detection layer flags the request, the attack event is written to the JSON log and the plaintext security log. The source IP receives a strike; if its cumulative strike count reaches the configured threshold, it is automatically added to the blacklist for the configured ban duration. A **403 Forbidden** response is returned.
9. **Reverse Proxy Forwarding.** Requests that pass all preceding checks are forwarded to the backend application by `proxy.py`. The backend's response is cleaned of hop-by-hop headers and returned to the client.

Architecture Diagram



Conclusion

Part 5 concludes the FlaskGuard series by supplying the two operational components that transform the security inspection engine into a deployable system. The reverse proxy provides transparent, standards-compliant forwarding of approved traffic to the protected backend, with correct header management, graceful error handling, and architecturally sound redirect behaviour. The demo backend gives the system a realistic test target and a self-contained demonstration environment, while its Docker network configuration faithfully models the intended production topology. The test suite validates every layer of the system — detection rules, file scanning, IP management, proxy behaviour, and administrative API — through a combination of isolated unit tests and full-pipeline integration tests.

Taken together across all five parts, FlaskGuard represents a thorough and pedagogically coherent implementation of a Web Application Firewall. It covers the complete engineering surface of WAF design: protocol-level request handling, pattern-based and machine-learning detection, file upload security, IP reputation management, structured attack logging, containerised deployment, and administrative tooling. While it is explicitly positioned as a learning and prototyping artefact rather than an enterprise-grade product, it successfully demonstrates every foundational concept that underpins commercial WAF solutions — in a codebase that is readable, modifiable, and fully tested.